# Unidad didáctica

## Bases de datos relacionales



# Conectores

Los **sistemas gestores de bases de datos** (**SGBD**) de distintos tipos tienen sus propios lenguajes especializados para operar con los datos que almacenan.

Para que los programas puedan interactuar con los SGBD se necesitan mecanismos que permitan a los programas de aplicación comunicarse con las bases de datos en estos lenguajes. Estos se implementan en API y se denominan **conectores**.



# Conectores para bases de datos relacionales

Los sistemas de bases de datos más utilizados hoy en día son los relacionales. Utilizan SQL como lenguaje estándar, aunque cada una tiene su propia versión de SQL con sus propias particularidades.

El uso de *drivers* permite desarrollar una arquitectura genérica en la que el conector tiene una interfaz común tanto para las aplicaciones como para las bases de datos.

De esta manera, el conector no es únicamente una API, sino una arquitectura, porque especifica unas interfaces que los distintos *drivers* tienen que implementar para acceder a las bases de datos correspondientes.

# Consultas sobre bases de datos

Los conectores permiten realizar todo tipo de operaciones sobre una base de datos relacional. En el caso de las bases de datos relacionales, la estructura de datos fundamental para el almacenamiento de la información es la **tabla**.

Una consulta de SQL devuelve un conjunto de filas o *recordset*. Los conectores permiten recuperar estos resultados fila a fila, mediante un objeto que actúa como iterador o cursor.

```
Connection c = getConnection(datos de conexión);
Statement s = c.createStatement();
ResultSet rs = s.executeQuery("SELECT ...");
while (rs.next()) { // En rs están disponibles más resultados de la consulta
        String dato1 = rs.getString(1); // Obtener String de primera columna
        int dato2 = rs.getInt(2); // Obtener int de segunda columna
}
s.close();
c.close();
```

# Consultas sobre bases de datos

```
Connection c = getConnection(datos de conexión);
Statement s = c.createStatement();
ResultSet rs = s.executeQuery("SELECT ...");
while (rs.next()) { // En rs están disponibles más resultados de la consulta
        String dato1 = rs.getString(1); // Obtener String de primera columna
        int dato2 = rs.getInt(2); // Obtener int de segunda columna
}
s.close();
c.close();
```

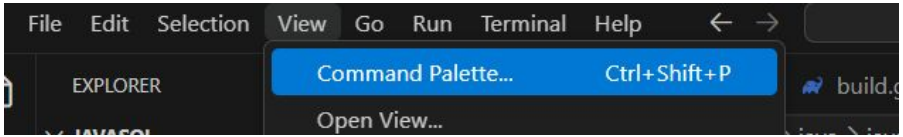
El objeto de tipo *ResultSet* actúa como iterador sobre los resultados de la consulta, que son un conjunto de filas. Una vez recuperada una fila se puede acceder a cada dato indicando su posición o su nombre.

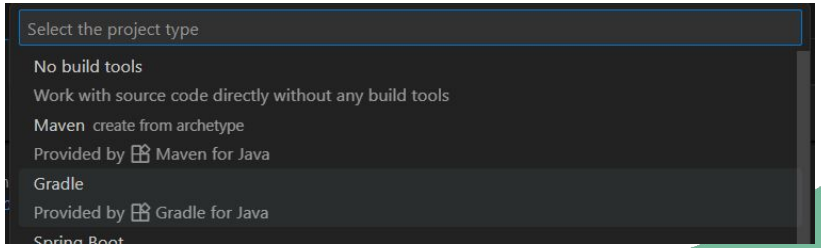
Este modelo de acceso es válido también para otros tipos de bases de datos, como bases de datos de objetos y de XML. Se trata siempre de abrir una conexión, realizar una consulta y obtener los resultados.

# Java Database Connectivity

La arquitectura de **conectores JDBC** para Java está basada en *drivers* y su API está disponible en el paquete *java.sql*. Los *drivers* de JDBC proporcionan clases que implementan las interfaces de la API JDBC para una base de datos particular.

Para poder utilizar JDBC, primero se crea un nuevo proyecto Java con *gradle*. En el ejemplo se utilizará Visual Studio Code.

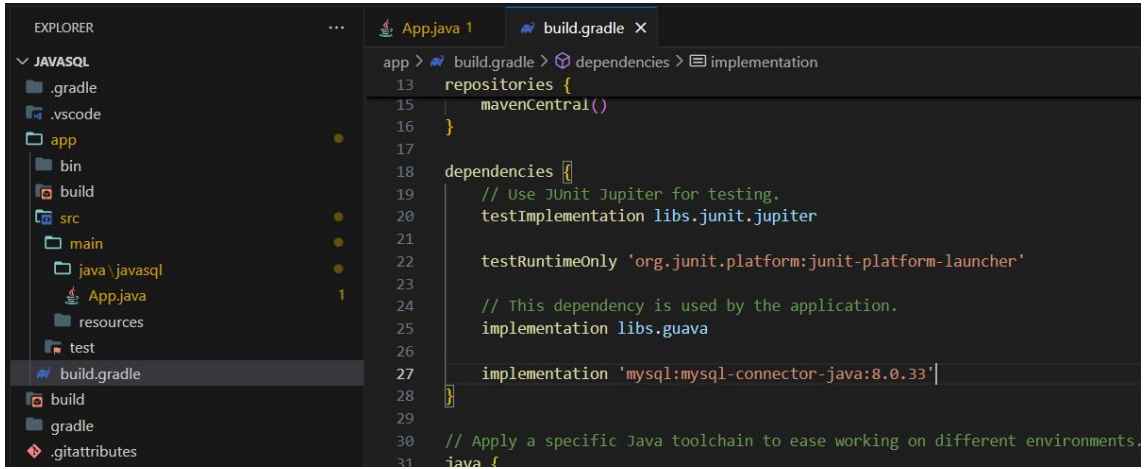


# Java Database Connectivity

Una vez creado el proyecto, se debe añadir el conector JDBC como dependencia en el archivo build.gradle y recompilar el proyecto para que descargue de forma automática el *driver* que realizará la conexión con la base de datos.

```
implementation 'mysql:mysql-connector-java:8.0.33'
```



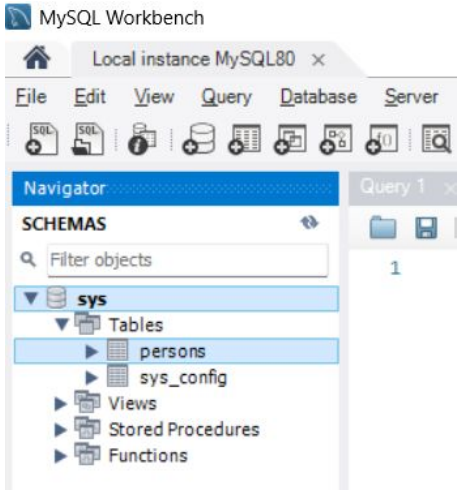
# Opción 1: Base de datos local

En MySQL Workbench, se crea una nueva instancia de servidor para almacenar la base de datos local a utilizar.



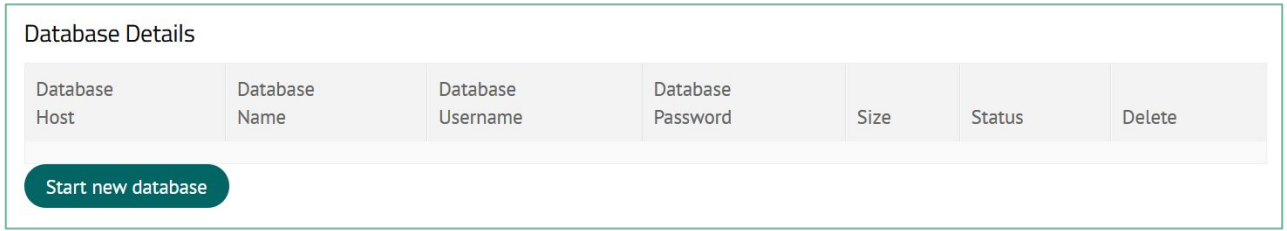
# Opción 1: Base de datos local

Dentro de MySQL Workbench, se crea una tabla para realizar la prueba de conexión. El nombre del *schema*, sys, se deberá escribir en el código Java para poder conectarse correctamente a la base de datos.

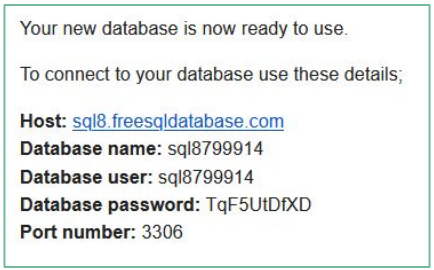


# Opción 2: Base de datos en la nube

Para tener una base de datos SQL en la nube se puede utilizar Free SQL Database. Para ello hay que crearse una cuenta en la web y seguir los pasos indicados. Una vez hecho esto, se crea una nueva base de datos pulsando en el botón *"Start new database"*:



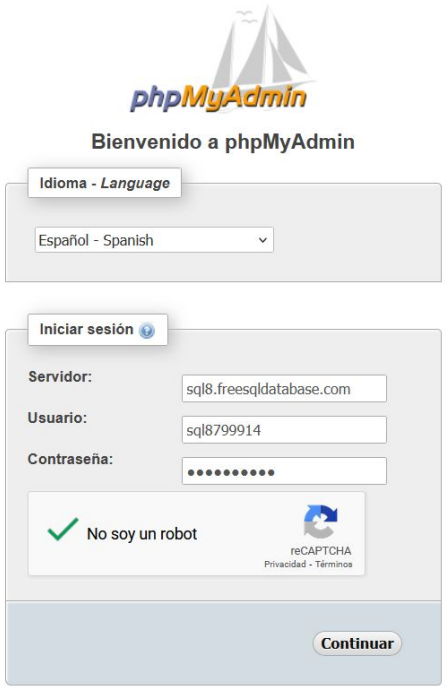

En unos minutos, llegarán al correo electrónico los datos de acceso a la base de datos:



# Opción 2: Base de datos en la nube

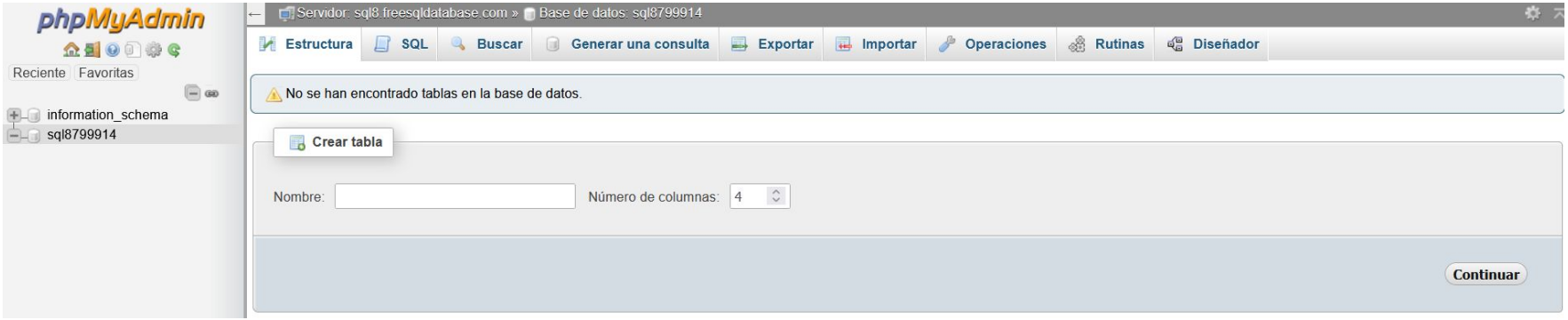
Para acceder al visor de la base de datos se utiliza phpMyAdmin, una herramienta muy útil para poder manejar tanto de forma visual como mediante consultas SQL las tablas de la base de datos.

Para iniciar sesión se escriben los datos de acceso obtenidos del paso anterior.



# Opción 2: Base de datos en la nube

Una vez iniciada la sesión, se procede a crear las tablas correspondientes y a la inserción de datos.



# Java Database Connectivity

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class Main {
    public static void main(String[] args) {
        String urlConnection = "jdbc:mysql://localhost:3306/sys";
        String user = "root";
        String password = "1234";

        try {
            Connection c = DriverManager.getConnection(urlConnection, user, password);
            Statement s = c.createStatement();
            ResultSet rs = s.executeQuery("SELECT * FROM persons");
            while (rs.next()) { // En rs están disponibles más resultados de la consulta
                String dato1 = rs.getString(1); // Obtener String de primera columna
                int dato2 = rs.getInt(2); // Obtener int de segunda columna

                System.out.println(dato1 + ", " + dato2);
            }

            s.close();
            c.close();
        } catch (Exception e) {
            System.err.println(e);
        }
    }
}
```

## Opción 2: en la nube

```java
String urlConnection = "jdbc:mysql://sql8.freesqldatabase.com:3306/sql8799914";
String user = "sql8799914";
String password = "TqF5UtDfXD";
```

*URL: jdbc:mysql://**host:port/database_name***

Para ejecutar una conexión y obtener datos con una consulta SELECT, se crea una clase Main en la ruta del proyecto /src/main/java/nombreproyecto/ y se añade el siguiente código:

# Ejercicio entregable #5

Crea un programa en Java que se conecte a una base de datos MySQL usando JDBC y realice consultas sobre una tabla creada previamente en MySQL Workbench.

1. Crea una base de datos que tenga una tabla llamada alumnos con las columnas id, nombre, edad y curso. Inserta varios registros en la tabla.
2. Escribe un programa en Java que muestre por consola los resultados de una consulta para obtener todas las filas de la base de datos anterior.

Sube únicamente el proyecto Java en formato zip a Aules.

# Ejercicio entregable #6

Crea un programa en Java que se conecte a una base de datos MySQL usando JDBC y realice consultas sobre la tabla creada en el ejercicio anterior. Escribe un programa en Java que muestre un menú en consola con las siguientes opciones:

1. Mostrar todos los alumnos
2. Mostrar alumnos por curso
3. Mostrar alumnos mayores de una edad
4. Salir

El usuario podrá seleccionar una opción y el programa ejecutará la consulta correspondiente mostrando los resultados por consola.

# Java Database Connectivity

Otra forma de obtener las columnas de una fila de la tabla es especificando directamente su nombre. Hay distintas funciones para distintos tipos: *getInt()*, *getString()*, *getDate()*, etc.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class Main {
    public static void main(String[] args) {
        String urlConnection = "jdbc:mysql://localhost:3306/sys";
        String user = "root";
        String password = "1234";

        try {
            Connection c = DriverManager.getConnection(urlConnection, user, password);
            Statement s = c.createStatement();
            ResultSet rs = s.executeQuery("SELECT * FROM persons");
            int i = 1;
            while (rs.next()) { // En rs están disponibles más resultados de la consulta
                System.out.println("Persona " + i++);
                System.out.println("DNI: " + rs.getString("DNI"));
                System.out.println("Apellidos: " + rs.getString("APELLIDOS"));
                System.out.println("CP: " + rs.getString("CP"));
            }

            s.close();
            c.close();
        } catch (Exception e) {
            System.err.println(e);
        }
    }
}
```

# Sentencias preparadas

En una aplicación real suele ser necesario ejecutar sentencias de SQL en las que intervengan variables, bien porque vengan introducidos desde la interfaz de usuario o porque dependan de un dato obtenido desde una fuente datos.

Las **sentencias preparadas** evitan problemas de seguridad y de rendimiento, ya que incluyen marcadores en determinados lugares de la sentencia donde van valores que se proporcionarán en el momento de ejecutar la sentencia.

Para hacer consultas con sentencias preparadas se utiliza la clase ***PreparedStatement***. A esta se le pasa como parámetro la sentencia SQL y se utiliza el carácter “?” como marcador.

# Sentencias preparadas

La función **executeQuery()** se usa para sentencias SQL de tipo SELECT. Devuelve un objeto ResultSet, que contiene las filas obtenidas de la consulta.

```
String sql = "SELECT id, nombre FROM usuarios WHERE activo = ?";
PreparedStatement ps = conn.prepareStatement(sql);
ps.setBoolean(1, true); // El primer parámetro indica la posición del ? en la sentencia y el segundo el valor

ResultSet rs = ps.executeQuery(); // devuelve las filas activas

while (rs.next()) {
   System.out.println(rs.getInt("id") + " - " + rs.getString("nombre"));
}
```

# Sentencias preparadas

La función **executeUpdate()** se usa para sentencias SQL de modificación de datos: *INSERT*, *UPDATE*, *DELETE* y también para *DDL* (ej. *CREATE TABLE*, *ALTER TABLE*, etc.). Devuelve un entero con el número de filas afectadas.

```
String sql = "UPDATE usuarios SET activo = ? WHERE id = ?";
PreparedStatement ps = conn.prepareStatement(sql);
ps.setBoolean(1, false);
ps.setInt(2, 10);

int filas = ps.executeUpdate(); // Devuelve cuántas filas fueron modificadas
System.out.println("Filas actualizadas: " + filas);
```

# Ejercicio entregable #7

Se desea desarrollar una aplicación en Java que utilice JDBC para conectarse a una base de datos MySQL. El programa deberá cumplir con los siguientes requisitos:

- Conexión a la base de datos: mostrar un mensaje de éxito o error al realizar la conexión.
- Crear una tabla llamada Animales con los siguientes campos: id, nombre, especie y edad.
- Crear un menú principal para el usuario que incluya las siguientes opciones:
  - Opción 1: pedir al usuario los datos de un animal y guardarlos en la tabla.
  - Opción 2: mostrar todos los registros de la tabla por la consola.
  - Opción 3: terminar el programa.

# Ejercicio entregable #8

Se desea desarrollar una aplicación en Java que utilice JDBC para conectarse a una base de datos MySQL. El programa gestionará una base de datos de series de televisión. El programa deberá cumplir con los siguientes requisitos:

- Conexión a la base de datos: mostrar un mensaje en consola indicando si la conexión se realizó correctamente.
- Creación de la tabla Series con los siguientes campos: id (entero, clave primaria, autoincremental), título, género, num_temporadas y finalizada (booleano).
- El programa deberá mostrar un menú en consola con las siguientes opciones:
  - 1. Insertar una nueva serie
  - 2. Listar todas las series
  - 3. Buscar series por género
  - 4. Mostrar series en emisión
  - 5. Actualizar número de temporadas de una serie
  - 6. Eliminar una serie por id
  - 7. Salir

# Ejercicio entregable #9

Desarrollar una aplicación en Java que utilice JDBC para conectarse a una base de datos MySQL. El programa gestionará una base de datos de libros y préstamos de una biblioteca. Los requisitos son los siguientes:

- Creación de tablas:
  - Tabla Libros:  id (INT, PK, autoincremental) título, autor, año de publicación y disponibilidad.
  - Tabla Usuarios: id (INT, PK, autoincremental), nombre y email.
  - Tabla Prestamos: id (INT, PK, autoincremental), id_usuario (FK), id_libro (FK), fecha del préstamo y fecha de devolución (puede ser null si no se ha devuelto).
- El programa deberá mostrar un menú con las siguientes opciones:
  - 1. Insertar un nuevo libro, 2. Listar todos los libros, 3. Buscar libros por autor, 4. Registrar un nuevo usuario, 5. Listar todos los usuarios, 6. Registrar un préstamo, 7. Registrar la devolución de un libro, 8. Mostrar préstamos activos, 9. Eliminar un libro por id, 10. Salir

# Resultados a objetos de clase

Cuando se hace una consulta SELECT, JDBC devuelve un *ResultSet*. Es como un cursor que apunta a las filas del resultado, de una en una. Un ResultSet no es una lista ni un objeto del modelo de datos: sólo sirve para leer los datos y se cierra con la conexión.

Lo habitual en estos casos es **transformar los datos en objetos** de la clase correspondiente. Para esto se recorre el ResultSet fila a fila y, por cada fila, se crea un objeto de la clase correspondiente que se guardarán, por ejemplo, en una lista o un array.

Cada atributo de la clase se corresponde con una columna de la tabla, por lo que los tipos de ambas deben ser compatibles.

# Resultados a objetos de clase: Ejemplo

```
public class Fruta {
   private int id;
   private String nombre;
   private double precio;
   private int stock;

   public Fruta(int id, String nombre, double precio, int stock) {
      this.id = id;
      this.nombre = nombre;
      this.precio = precio;
      this.stock = stock;
   }

   public int getId() { return id; }
   public void setId(int id) { this.id = id; }
   public String getNombre() { return nombre; }
   public void setNombre(String nombre) { this.nombre = nombre; }
   public double getPrecio() { return precio; }
   public void setPrecio(double precio) { this.precio = precio; }
   public int getStock() { return stock; }
   public void setStock(int stock) { this.stock = stock; }
}
```

```
CREATE TABLE FRUTAS (
   id  INT PRIMARY KEY,
   nombre  VARCHAR(50) NOT NULL,
   precio  DECIMAL(6,2),
   stock   INT
);
```

```
List<Fruta> frutas = new ArrayList<>();
String sql = "SELECT id, nombre, precio, stock FROM FRUTAS";
PreparedStatement ps = con.prepareStatement(sql);
ResultSet rs = ps.executeQuery();
try {

      while (rs.next()) {
         Fruta f = new Fruta();
         f.setId(rs.getInt("id"));
         f.setNombre(rs.getString("nombre"));
         f.setPrecio(rs.getDouble("precio"));
         f.setStock(rs.getInt("stock"));
         frutas.add(f);
      }
}
return frutas;
```

# Ejercicio entregable #10

Se pide desarrollar una aplicación en Java que utilice JDBC para conectarse a una base de datos MySQL y trabajar con los datos de un concesionario de coches. El programa debe obtener los datos de la tabla Coches (id, marca, modelo, anyo, precio y kilometros) y guardarlos en una lista de objetos Coche. Además, se pide:

1. Recorrer la lista de objetos Coche y mostrar por la terminal la información de todos los coches que sean posteriores al año 2005.
2. El precio medio de todos los coches, el coche más caro y el coche más barato, mostrando marca, modelo y precio.

# Ejercicio entregable #11

Se pide desarrollar una aplicación en Java que utilice JDBC para conectarse a una base de datos MySQL y trabajar con los datos de una tienda de videojuegos. El programa debe obtener los datos de la tabla Videojuegos (id, titulo, plataforma, genero, anyo, precio y valoracion) y guardarlos en una lista de objetos Videojuego. Además, se pide:

1. Recorrer la lista de objetos y mostrar por la terminal el título, la plataforma y la valoración de todos los videojuegos que tengan una valoración igual o superior a 8 y un precio inferior a 30 €.
2. Calcular y mostrar por la terminal el número de videojuegos que hay de cada plataforma.